

Technical Notes: Depth-Based Student Distillation for Quadruped Parkour on Genesis

LeggedGym-Ex Repository Status Report

Auto-generated Research Note

March 25, 2026

1 Introduction and Goals

This document records the current state of the **LeggedGym-Ex** repository, which implements a two-phase teacher-student training pipeline for quadruped parkour locomotion on the Unitree Go2 robot, using the Genesis physics simulator.

End goal. Train a student policy that receives *monocular RGB* images, runs them through a depth-estimation network (e.g. Depth Anything V2), and feeds the inferred depth map to the policy—enabling deployment without a physical depth camera.

Current milestone. A *ground-truth depth* student (simulator depth camera) has been trained to convergence; a *depth-estimation* student (RGB + Depth Anything V2 Small, inferred depth normalized to the student tensor) is trained with the same distillation pipeline. Offline ablations (see §??) indicate that the depth-estimation checkpoint tested relies on the depth stream, not proprioception alone.

Reference work. The architecture and training procedure follow Cheng et al. “Extreme Parkour with Legged Robots” (arXiv:2309.14341) [?], with adaptations for the Genesis simulator and Go2 robot morphology.

2 System Overview

2.1 Hardware and Simulator

- **GPU:** $4 \times$ NVIDIA L40S (45 GB each).
- **Simulator:** Genesis (GPU-accelerated rigid-body physics with batch depth rendering via CUDA scene cameras on Linux).
- **Robot:** Unitree Go2 (12 DoF, 4 legs $\times$ 3 joints).

2.2 Two-Phase Training Pipeline

1. **Phase 1 — Teacher (PPO):** A privileged teacher policy observes proprioception *plus* 132 ray-cast scandot heights and is trained with PPO on a parkour curriculum (stairs, hurdle blocks, gaps). The critic additionally observes base linear velocity, dynamics parameters, and contact states.
2. **Phase 2 — Student (Dagger distillation):** A student policy observes only proprioception (53-dim) and a single 58×87 depth image. A CNN-GRU depth encoder produces a 32-dim terrain latent and a 2-dim yaw estimate. The student’s actor MLP is initialized

from the teacher’s actor weights and fine-tuned via DAgger: at each step the frozen teacher provides target actions, and the student minimizes L2 action loss + L2 yaw loss.

3 Architecture Details

3.1 Observation Layout

The proprioceptive observation vector (53-dim) is ordered as:

Component	Description	Dims
Commands	goal vel, heading ($\sin \theta_e, \cos \theta_e$), goal speed	7
Gravity	projected gravity in body frame	3
Angular velocity	base angular velocity (scaled)	3
Joint positions	$q - q_{\text{default}}$ (scaled)	12
Joint velocities	$\dot{q}$ (scaled)	12
Previous actions	a_{t-1}	12
Foot contacts	binary contact indicators	4

The teacher additionally receives 132 scandot heights (12×11 grid) appended after the proprioceptive vector, for a total actor observation of 185 dims. The student’s actor only sees the 53-dim proprio; terrain perception comes exclusively through the depth encoder’s 32-dim latent.

Heading command indices 4–5 ($\sin \theta_e, \cos \theta_e$) are *masked to zero* in the depth encoder’s proprioceptive input, forcing the GRU to predict yaw from the depth stream.

3.2 Student Network Architecture

Module	Architecture
DepthBackbone	$\text{Conv2d}(1, 32, k=5) \rightarrow \text{MaxPool}(2) \rightarrow \text{ELU} \rightarrow \text{Conv2d}(32, 64, k=3) \rightarrow \text{ELU} \rightarrow \text{FC}(128) \rightarrow$
CombinationMLP	$\text{FC}(32+53, 128) \rightarrow \text{ELU} \rightarrow \text{FC}(128, 32)$
GRU	$\text{GRU}(\text{input}=32, \text{hidden}=512, 1 \text{ layer})$
OutputMLP	$\text{FC}(512, 34) \rightarrow \text{Tanh}$
Actor	$\text{FC}(53+32, 512) \rightarrow \text{ELU} \rightarrow \text{FC}(256) \rightarrow \text{ELU} \rightarrow \text{FC}(128) \rightarrow \text{ELU} \rightarrow \text{FC}(12) \rightarrow \text{Hardtan}$

The Tanh output is split into a 32-dim *latent* (fed to the actor alongside proprio) and a 2-dim *raw yaw* estimate, scaled by $1.5\times$ to match oracle heading magnitude.

3.3 Mixture of Teacher and Student (MTS)

Following the reference, a soft gating mechanism replaces the student’s heading observation with its own yaw prediction only when the prediction is close to the oracle:

$$\mathbf{o}_{\text{heading}} = \begin{cases} \hat{\mathbf{y}}_{\text{student}} & \text{if } \|\hat{\mathbf{y}}_{\text{student}} - \mathbf{y}_{\text{oracle}}\| < \tau \\ \mathbf{y}_{\text{oracle}} & \text{otherwise} \end{cases}$$

with threshold $\tau = 0.6$. This prevents early-training distribution shift while encouraging the student to eventually rely on its own yaw prediction.

4 Differences from Canonical Implementations

Table ?? summarizes the key differences between our implementation and the reference repositories.

Table 1: Comparison with reference implementations.

Aspect	Extreme Parkour (Isaac-Gym)	Ours (Genesis)
Simulator	IsaacGym (NVIDIA)	Genesis (open-source, CUDA batch cameras)
BPTT strategy	Full 120-step, single backward()	Windowed BPTT(24), 5 windows per rollout
num_envs (student)	192	48 (Genesis depth cameras are memory-intensive)
Latent loss	Not used	Not used (removed; was present in initial buggy code)
Depth noise	Per-pixel (dis_noise) Gaussian	Scalar uniform noise per frame (simplified)
Depth normalization	[0, 1] range	[-0.5, +0.5] range
Depth resolution	58 × 87	58 × 87 (identical after crop/resize)
Teacher critic	Same as ours	Proprio + scandots + base lin vel + dynamics + contacts
Terrain families	Parkour-specific obstacles	Stairs, hurdle blocks, gaps (3 families × 4 variants)
Curriculum	Row-based difficulty	Row-based difficulty (identical scheme)
GRU reset on done	No reset during training	No reset during training (matching reference)

Why windowed BPTT instead of full BPTT. Genesis batch depth rendering allocates substantially more GPU memory per environment than IsaacGym’s tensor-based API. With 48 environments, the simulator alone consumes ~ 11 GB on an L40S. Full BPTT through 120 CNN+GRU steps requires retaining the entire computation graph, which caused silent CUDA hangs when attempted (the process continued consuming CPU/GPU resources but produced no output and never completed the backward pass). A 24-step window (covering ~ 5 depth camera updates at decimation=5) provides meaningful temporal gradient flow while bounding peak memory.

Why 48 environments. The reference uses 192 environments on IsaacGym. Initial attempts with 192 environments on Genesis caused immediate OOM during the first CNN forward pass (44 GB GPU fully consumed by simulator state alone). Reducing to 48 environments leaves sufficient headroom for the BPTT computation graph.

5 Bugs Found and Fixed

The initial codebase had five bugs preventing student convergence, discovered through careful comparison with the reference implementation.

5.1 Bug 1 (Critical): Per-Step Backward Destroyed GRU BPTT

The original `ParkourDistillation.accumulate_step()` called `backward()` at *each* of the 120 rollout steps, followed by `detach_gru_hidden()`. This is truncated BPTT(1)—the GRU receives zero temporal gradient and cannot learn any sequential dependencies.

Fix: Rewrote the distillation algorithm to collect live-graph tensors across a configurable window (`bptt_window=24`), compute loss once per window, and call `backward()` at window boundaries. The GRU hidden state is detached only after each window’s backward pass. Gradients accumulate across all 5 windows, with a single optimizer step after the full 120-step rollout.

5.2 Bug 2 (High): Teacher Observations Lacked Noise

The student environment applied observation noise to `obs_buf` (the student’s proprioceptive input) but constructed `teacher_actor_obs_buf` from clean sensor readings. Since the teacher was *trained with noise*, querying it with clean observations produced misaligned target actions.

Fix: Applied the same noise realization to both the student’s proprioceptive observation and the proprioceptive portion of the teacher’s observation. Scandot noise (matching the teacher’s training noise scale) is also applied to the teacher’s scandot portion.

5.3 Bug 3 (Medium): Latent Loss with Range Mismatch

The distillation loss included an L2 term between the student’s depth latent (Tanh, range $[-1, 1]$) and the teacher’s scandot latent (ELU, range $(-1, +\infty)$). The reference uses *only* action loss + yaw loss. The range mismatch meant the student could never perfectly match teacher latents, creating a persistent conflicting gradient.

Fix: Removed the latent loss entirely. The student is trained only on action matching and yaw matching, consistent with the reference.

5.4 Bug 4 (Medium): Excessive Environment Count

The student config specified `num_envs=2048`, inherited from the teacher. With full BPTT (or even windowed BPTT), this caused OOM since Genesis depth cameras consume significant GPU memory per environment.

Fix: Reduced to 48 environments (the maximum that fits on one L40S with comfortable headroom for the backward pass).

5.5 Bug 5 (Low): Missing Depth Noise

The reference applies Gaussian noise (`dis_noise`) to depth images for sim-to-real robustness. The initial codebase had no depth augmentation.

Fix: Added configurable depth noise (`depth_noise_level=0.1`) applied as a scalar uniform perturbation after sensor rendering.

5.6 Additional Fix: In-Place Depth Buffer Modification

During full BPTT development, the backward pass failed with “`RuntimeError: variable has been modified by an inplace operation`” because the simulator’s depth buffer is modified in-place between steps by `update_sensors()`. The student’s `forward_depth()` received a reference to the same buffer at each step, but autograd expected the original tensor.

Fix: Clone the depth tensor before each forward pass: `student_depth.clone()`.

6 Training Results

6.1 Teacher Training

The teacher was trained for 2500 iterations (4096 envs, PPO, ~2 hours on one L40S). Table ?? shows the convergence trajectory.

Table 2: Teacher training metrics (PPO, 4096 envs).

Iter	Reward	Success	Progress	Terrain Level
0	-1.1	0.000	0.008	0.00
500	51.4	0.827	0.907	2.03
1000	52.9	0.791	0.908	1.99
1500	57.5	0.685	0.937	2.03
2000	74.4	0.225	0.926	1.99

The teacher reached 83% success rate at peak (iter 500), with mean reward climbing to 74 as the curriculum pushed harder terrain. Progress ratio stabilized above 90%.

6.2 Student Training — Run 1 (Iterations 0–1003)

The student was trained with windowed BPTT(24), 48 envs, 120 steps/env, lr= 10^{-3} , on one L40S GPU. Table ?? shows the convergence.

Table 3: Student Run 1 metrics (Dagger distillation, 48 envs, windowed BPTT).

Iter	Act. Loss	Yaw	Reward	Success	Progress	Level	Row	WPs	Ep. Len
0	1.562	0.284	4.3	0.000	0.062	0.00	0.00	0.0	50
100	0.748	0.108	46.9	0.092	0.710	1.00	0.98	1.5	549
200	0.477	0.065	72.6	0.000	0.627	1.47	1.43	1.8	773
300	0.425	0.052	82.7	0.471	0.939	2.04	1.91	3.0	861
500	0.403	0.041	86.6	0.025	0.811	2.00	1.65	1.7	901
700	0.313	0.038	88.9	0.142	0.925	1.73	2.06	2.8	928
800	0.324	0.031	92.7	0.100	0.949	2.09	1.28	2.2	959
1000	0.329	0.041	89.5	0.358	0.946	1.91	2.15	3.0	928

Key observations.

- Action loss converged from 1.56 to ~ 0.33 ($5\times$ reduction).
- Yaw loss converged from 0.28 to ~ 0.03 ($10\times$ reduction).
- Mean reward reached 92.7 at iter 800, *exceeding* the teacher’s typical 74 reward—likely because the student’s depth perception provides smoother terrain information than the teacher’s sparse scandots.
- Peak success rate of 47% (iter 300) and 80% (measured via TensorBoard max) with progress ratio consistently above 94%.
- Terrain curriculum advanced to level ~ 2.0 (out of 4 rows), with the student completing nearly full courses on medium-difficulty terrain.

6.3 Student Training — Run 2 (Resumed, Iterations 1003–1492)

After resuming from the Run 1 checkpoint, the student continued improving. Table ?? shows metrics relative to the resumed run’s local iteration counter.

Action loss continued decreasing to 0.27, yaw loss to 0.017, and reward stabilized in the 92–94 range. The terrain level advanced to ~ 2.2 . The training is still running as of this writing.

6.4 Depth-Estimation Student (RGB $\rightarrow$ DA-v2 $\rightarrow$ policy)

The task `go2_parkour_depth_est_student` disables the simulator depth camera (`add_depth=false`), renders RGB at the same pose as the old depth camera (`BatchRenderer`), and runs Depth Any-

Table 4: Student Run 2 (resumed from iter 1003).

Δ Iter	Act. Loss	Yaw	Reward	Success	Progress
0	0.346	0.024	2.7	0.000	0.057
100	0.314	0.028	92.1	0.158	0.954
200	0.286	0.025	92.5	0.017	0.974
300	0.292	0.030	93.0	0.067	0.944
400	0.316	0.027	93.5	0.083	0.939
489	0.272	0.017	92.8	0.000	0.918

thing V2 Small on the batch. Inferred depth is resized to 58×87 and per-frame min-max normalized to $[-0.5, 0.5]$ before the CNN-GRU encoder. The student network and losses are unchanged from the ground-truth depth student.

Training vs. teacher vs. GT-depth student. Table ?? compares *representative* training snapshots. Rewards are environment returns under the same parkour reward function, but iterations are not aligned across curriculum phases (success is also noisy at 48 envs). The teacher uses privileged scandots; both students use proprioception + depth (simulated or inferred).

Table 5: Representative training metrics: teacher (PPO) vs. ground-truth depth student vs. depth-estimation student (DAgger).

Policy	Algorithm	Terrain obs.	Iter	Reward	Success	Progress
Teacher	PPO	Scandots (priv.)	2000	74.4	0.23	0.93
GT depth student	DAgger	Sim. depth camera	800	92.7	0.10	0.95
GT depth student	DAgger	Sim. depth camera	1492	92.8	0.00	0.92
DA-v2 student	DAgger	Inferred depth	$\sim 310^\dagger$	72.2	0.00	0.74
DA-v2 student	DAgger	Inferred depth	$\sim 540^\ddagger$	52.2	0.00	0.72

[†]Resumed run; console snapshot (iteration ~ 310): mean reward 72.2, action loss ~ 0.24 , yaw loss ~ 0.01 . [‡]Earlier run on GPU 1 (iteration ~ 538): mean reward 52.2, action loss ~ 0.46 , mean episode length ~ 589 ; success rate is batch-noisy at 48 envs.

Interpretation. The teacher reaches high success when the curriculum is favorable (e.g. 83% at iter 500 in Table ??); later iterations trade success for harder terrain. The GT-depth student matches or exceeds teacher *progress* at strong checkpoints (action loss ~ 0.27 – 0.33) with the usual success-rate variance. The DA-v2 student is slower per iteration (RGB + estimator overhead) and, at comparable mid-training snapshots, shows lower mean reward and sparser success than the best GT-depth student—consistent with noisier exteroceptive input. Continued training and evaluation on a fixed checkpoint are needed for a fair final comparison.

6.5 Ablation: Does the depth-estimation student use depth?

We evaluated the frozen policy with `legged_gym/scripts/ablate_parkour_student_depth.py` (headless rollouts, ~ 96 completed episodes per condition, same seed and terrain). The `apply_student_depth_abl` utility replaces `student_depth` before `act_inference`: *none* (real depth), *zero* (zeros), *noise* (i.i.d. uniform in $[-0.5, 0.5]$), *shuffle* (permute depth across parallel envs).

Checkpoint. `logs/go2_parkour_depth_est_student/Mar25_15-12-01_depth_est_student_genesis/mo` (latest checkpoint in that run directory at the time of the study).

Table 6: Depth ablation (eval rollouts, ~ 96 – 98 episodes per row).

Ablation	Success rate	Mean progress	Mean WPs
none (real inferred depth)	0.031	0.928	1.00
zero	0.010	0.592	0.63
noise	0.061	0.880	0.98
shuffle	0.010	0.889	0.94

Takeaway. *Zero* depth sharply reduces mean progress along the course and waypoints reached versus *none*, indicating the policy is not behaving as proprioception-only locomotion at this checkpoint. *Shuffle* keeps high progress but collapses success to $\sim 1\%$, consistent with wrong depth-to-scene correspondence while still moving under commands and recurrent state. *Noise* remains closer to *none* on progress (random frames still carry spatial structure); with only ~ 100 episodes, success rates are high-variance.

7 Open Issues and Next Steps

7.1 Remaining Issues

1. **Terminal output buffering.** Console output (stdout) stops being captured after ~ 500 iterations, though the training process continues running normally. TensorBoard events and checkpoints are written correctly. This appears to be a Python stdout buffering issue when output is redirected to a pipe. Workaround: monitor via TensorBoard or periodic checkpoint inspection.
2. **Depth noise is scalar, not per-pixel.** The current depth noise augmentation applies a single random scalar offset to the entire depth image, rather than per-pixel Gaussian noise as in the reference. This provides less diversity for sim-to-real robustness. The per-pixel version was deferred to avoid additional GPU memory overhead during BPTT.
3. **Low environment count (48 vs. 192).** Genesis’s memory-intensive depth rendering limits us to 48 parallel environments, compared to 192 in the reference. This reduces sample diversity per iteration and may slow convergence. Potential mitigations: (a) gradient checkpointing through the CNN to reduce activation memory, (b) using Genesis’s lower-level API to share rendering resources, (c) mixed-precision training (FP16 activations).
4. **Success rate instability.** Success rate oscillates between 0% and 47% across iterations due to the small number of episodes completed per iteration ($48 \text{ envs} \times \sim 0.13 \text{ episodes/env/iter} \approx 6 \text{ episodes}$). With only 6 Bernoulli samples, the variance is inherently high. The progress ratio (0.94–0.97) is a more stable convergence indicator.
5. **Resume path resolution.** The `-resume` flag resolves `load_run=-1` by sorting all directories in the log root alphabetically and selecting the last one. A `datasets/` directory (used for demo replay) sorts after the timestamped run directories (e.g. `Mar24_...`), causing resume to fail. Fix: exclude non-run directories or specify `-load_run` explicitly.

7.2 Planned Next Steps

1. **Complete 5000-iteration training** and evaluate final success rate and progress ratio on the hardest terrain rows.
2. **Per-pixel depth noise.** Replace scalar noise with per-pixel Gaussian noise ($\sigma = 0.1$) matching the reference’s `dis_noise` implementation.

3. **Depth estimation network integration (implemented).** The `go2_parkour_depth_est_student` task uses RGB (BatchRenderer) + Depth Anything V2 Small with per-frame min-max normalization to $[-0.5, 0.5]$. Remaining work: longer training to match GT-depth student checkpoints, optional metric-depth calibration, and per-pixel noise for robustness.
4. **Evaluate sim-to-real gap.** Deploy the trained student on physical Go2 hardware with an onboard RGB camera and assess transfer quality.

8 Hyperparameter Summary

Table 7: Key hyperparameters.

	Parameter	Teacher	Student
4*Env	<code>num_envs</code>	4096	48
	<code>episode_length_s</code>	20	20
	<code>num_steps_per_env</code>	48	120
	Depth camera decimation	—	5
4*Training	Algorithm	PPO	DAGger (windowed BPTT)
	Learning rate	10^{-3}	10^{-3}
	<code>max_grad_norm</code>	1.0	1.0
	<code>bptt_window</code>	—	24
3*Loss	Action loss coef	—	1.0
	Yaw loss coef	—	1.0
	MTS yaw threshold	—	0.6
2*Noise	Observation noise	Yes	Yes (matched to teacher)
	Depth noise level	—	0.1
2*Terrain	Families	stairs, hurdle_block, gap	
	Rows $\times$ Cols	4×12 (4 difficulty $\times$ 3 families $\times$ 4 variants)	

9 Repository Structure (Relevant Files)

```

LeggedGym-Ex/
  legged_gym/
    envs/go2/
      go2_parkour_teacher/
        go2_parkour_teacher.py          # Teacher env (
          LeggedRobotParkour)
        go2_parkour_teacher_config.py   # Teacher config (PPO, 4096
          envs)
      go2_parkour_student/
        go2_parkour_student.py          # Student env (depth + proprio)
        go2_parkour_student_config.py   # Student config (DAGger, 48
          envs)
    envs/base/
      legged_robot_parkour.py           # Base parkour env with
        curriculum
      parkour_observation.py            # Observation spec (prop/actor/
        critic dims)
    simulator/

```



```

    genesis_simulator.py          # Genesis backend (depth
        rendering)
rsl_rl/
  algorithms/
    parkour_distillation.py      # Windowed BPTT DAgger
        algorithm
  runners/
    parkour_student_runner.py    # Student training loop
modules/
  actor_critic_parkour.py        # Teacher network (scandot
    encoder)
  actor_critic_parkour_student.py # Student network (CNN+GRU+
    actor)
logs/
  go2_parkour_teacher/          # Teacher checkpoints and TB
    events
  go2_parkour_student/          # GT-depth student checkpoints
    and TB
  go2_parkour_depth_est_student/ # DA-v2 depth-estimation
    student logs
legged_gym/scripts/
  ablate_parkour_student_depth.py # Headless depth ablation (none
    /zero/noise/shuffle)
legged_gym/utils/
  student_depth_ablation.py      # apply_student_depth_ablation
    (...)

```

References

- [1] X. Cheng, K. Shi, A. Agarwal, D. Pathak, “Extreme Parkour with Legged Robots,” *arXiv preprint arXiv:2309.14341*, 2023. Code: <https://github.com/chengxuxin/extreme-parkour>